Tugas Besar 1

Optimasi Kebijakan Suku Bunga Bank Indonesia berbasis *Local Search*

IF3170 Intelegensi Artifisial

Semester 1 Tahun 2027/2028



Disusun oleh:

Muhammad Aufar Rizqi Kusuma (13524061)

Laboratorium Intelegensi Buatan

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **2**
1.1 Pengantar Permasalahan . . . 2
1.2 Tujuan Permasalahan . . . 3
1.3 Kerangka Teoretis Model Makroekonomi . . . 3
1.3.1 Dynamic IS Curve (Sisi Permintaan) . . . 3
1.3.2 Hybrid New Keynesian Phillips Curve (Sisi Penawaran dan Inflasi) . . . 4
1.3.3 Uncovered Interest Rate Parity (Nilai Tukar) . . . 4
1.3.4 Tiga Persamaan Proksi . . . 5
1.3.5 Guncangan Stokastik $\epsilon_t^{(\cdot)}$ . . . 5
2 Spesifikasi Masalah **6**
2.1 Representasi State . . . 6
2.2 Aturan dan Batasan Permasalahan . . . 6
2.2.1 Definisi *Feasible* dan Peran π_T . . . 7
2.3 *Succesor Function* . . . 7
2.4 *Objective Function* . . . 8
2.4.1 Definisi *Objective Function* . . . 8
2.4.2 *Loss Function* Teoretik . . . 8
2.4.3 *Loss Function* Final . . . 9
2.5 Formulasi *Initial State* . . . 9
2.6 *Model Evaluation Metric* . . . 10
2.7 Daftar Command Utama . . . 11
2.8 Contoh Input dan Output . . . 14
3 Spesifikasi Bonus **18**
3.1 Interaksi Kebijakan Fiskal-Moneter (Stackelberg Game) . . . 18
3.1.1 Perluasan State . . . 18
3.1.2 Perluasan Model Makroekonomi . . . 18
3.1.3 Mekanisme Pencarian *Joint Equilibrium* . . . 19
3.2 Visualisasi *Local Search* . . . 19

Bab 1

Pendahuluan

1.1 Pengantar Permasalahan

Kebijakan moneter merupakan salah satu instrumen utama negara dalam menjaga stabilitas ekonomi. Di Indonesia, kewajiban Bank Indonesia (BI) sebagai otoritas moneter berakar dari UU No. 23 Tahun 1999 tentang Bank Indonesia yang telah diubah beberapa kali, dengan perubahan terakhir melalui UU No. 4 Tahun 2023 tentang Pengembangan dan Penguatan Sektor Keuangan (UU P2SK). Pasal 9 angka 2 UU P2SK merumuskan tujuan Bank Indonesia sebagai berikut.

> *Tujuan Bank Indonesia adalah mencapai stabilitas nilai rupiah, memelihara stabilitas Sistem Pembayaran, dan turut menjaga Stabilitas Sistem Keuangan dalam rangka mendukung pertumbuhan ekonomi yang berkelanjutan.*

Stabilitas nilai rupiah dalam mandat tersebut secara eksplisit mencakup kestabilan harga barang dan jasa (inflasi) serta kestabilan nilai tukar rupiah terhadap mata uang negara lain, yang operasionalnya diatur lebih lanjut dalam Peraturan Bank Indonesia (PBI) No. 11 Tahun 2024 tentang Pengendalian Moneter yang menetapkan suku bunga acuan BI-Rate sebagai sasaran operasional utama kebijakan moneter. Pada 4 Juni 2026, DPR RI mengesahkan revisi lanjutan UU P2SK yang memperluas mandat BI untuk turut mendukung pertumbuhan ekonomi dan penciptaan lapangan kerja di samping tugas stabilisasi yang sudah ada, sehingga kebijakan moneter BI tidak dapat lagi dirancang semata-mata untuk menekan inflasi tetapi harus mempertimbangkan dampaknya terhadap sektor riil dan daya beli masyarakat.

Mandat hukum di atas secara langsung menghasilkan sebuah *trade-off* multi-objektif yang harus diselesaikan Gubernur BI setiap periode Rapat Dewan Gubernur (RDG). Menaikkan suku bunga akan menekan inflasi dan memperkuat rupiah, tetapi menekan *output* dan daya beli serta memberatkan biaya utang pemerintah. Sebaliknya, menurunkan suku bunga akan mendorong *output* dan daya beli, tetapi berisiko melemahkan rupiah dan memicu *capital outflow*.



Gambar 1.1: Rapat Dewan Gubernur Bank Indonesia. Sumber

Sebagai ilustrasi kondisi riil yang dihadapi, pada RDG Juli 2026 BI mempertahankan BI-Rate di 5,75% di tengah inflasi tahunan 3,34% (naik dari 3,08% bulan sebelumnya), nilai tukar rupiah di sekitar Rp17.885 per USD yang melemah 7,29% *year-to-date*, serta Indonesia baru mengalami defisit transaksi berjalan pertama setelah 72 bulan surplus berturut-turut. Kondisi ini terjadi ketika Federal Reserve AS menahan Fed Funds Rate di kisaran 3,50% sampai 3,75%, sehingga selisih suku bunga BI-Rate terhadap suku bunga AS menjadi pertimbangan penting untuk mencegah arus keluar modal asing.

Dari uraian di atas, dapat terlihat bahwa keputusan suku bunga tidak berdiri sendiri per periode dan dampaknya menjalar melalui inflasi, *output*, dan nilai tukar ke periode-periode berikutnya. Menentukan satu angka BI-Rate untuk satu kuartal saja sudah cukup sulit karena melibatkan *trade-off* yang dijelaskan sebelumnya. Menentukan delapan angka sekaligus untuk dua tahun ke depan, dengan setiap keputusan saling memengaruhi melalui rantai kausal yang kompleks, jelas berada di luar kemampuan intuisi atau *rule of thumb* sederhana. Pendekatan naif seperti mematok suku bunga konstan atau mengikuti pola historis akan mengabaikan dinamika antar-periode yang non-linear.

Maka dari itu, permasalahan ini akan dicoba untuk direduksi menjadi sebuah persoalan *local search* yang mencari *complete-state* berupa konfigurasi suku bunga sepanjang delapan kuartal yang memaksimalkan kesejahteraan sosial. Dengan formulasi ini, penentuan BI-Rate tidak lagi bergantung pada intuisi atau penilaian subjektif pengambil kebijakan, melainkan dijalankan secara sistematis melalui eksplorasi ruang state yang dipandu oleh fungsi objektif kuantitatif. Harapannya, pendekatan ini dapat menjadi kerangka bantu keputusan yang lebih terstruktur dibandingkan metode konvensional yang bersifat *ad-hoc*.

1.2 Tujuan Permasalahan

Tujuan utama permasalahan ini adalah menentukan state suku bunga acuan $s = (r_1, r_2, \ldots, r_T)$ yang memaksimalkan kesejahteraan gabungan dari stabilitas harga, stabilitas nilai tukar, dan daya beli masyarakat. Horizon kebijakan yang digunakan adalah $T = 8$ periode kuartalan (dua tahun ke depan), dimulai dari kondisi aktual BI-Rate per Juli 2026. Tiap r_t harus berada dalam rentang 3,00% sampai 8,00% dengan kelipatan 25 basis poin sesuai konvensi RDG BI. State s harus memenuhi beberapa *constraint* yang mencerminkan kondisi neraca keuangan riil Indonesia. Secara formal, permasalahan dinyatakan sebagai pencarian state optimal pada ruang keadaan $\mathcal{S}$.

$$s^* = \arg\max_{s \in \mathcal{S}} J(s)$$

Fungsi objektif $J(s)$ merupakan negatif dari *quadratic loss function* yang mengukur total deviasi variabel makroekonomi dari targetnya sepanjang horizon T. Semakin mendekati nol nilai $J(s)$, semakin optimal state tersebut dalam menyeimbangkan inflasi, *output*, daya beli, dan stabilitas suku bunga.

1.3 Kerangka Teoretis Model Makroekonomi

Untuk mengevaluasi seberapa baik sebuah state suku bunga $s = (r_1, \ldots, r_T)$, diperlukan model yang mampu mensimulasikan bagaimana keputusan BI-Rate pada tiap kuartal menjalar menjadi inflasi, pertumbuhan ekonomi, dan pergerakan nilai tukar. Model yang digunakan mengacu pada kerangka *New Keynesian* untuk ekonomi terbuka kecil (*small open economy*), mengikuti Galí dan Monacelli [1] serta Clarida, Galí, dan Gertler [2]. Model ini terdiri dari tiga persamaan inti yang saling terhubung secara rekursif. *Output* hari ini dipengaruhi oleh suku bunga kemarin, inflasi hari ini dipengaruhi oleh *output* hari ini, dan nilai tukar hari ini dipengaruhi oleh selisih suku bunga domestik terhadap suku bunga Amerika Serikat. Dari ketiga persamaan inti tersebut, diturunkan tiga persamaan proksi tambahan yang merepresentasikan daya beli masyarakat, neraca transaksi berjalan, dan imbal hasil obligasi pemerintah. Seluruh persamaan beroperasi pada tiap periode t dari 1 sampai $T = 8$, dengan nilai awal periode $t = 0$.

1.3.1 Dynamic IS Curve (Sisi Permintaan)

Persamaan IS (*Investment-Saving*) menggambarkan hubungan antara suku bunga riil dan *output gap*, yaitu selisih antara *output* aktual dan *output* potensial suatu perekonomian.

$$y_t = \rho_y \, y_{t-1} - \beta \, (r_{t-1} - \pi_{t-1} - r^*) + \epsilon_t^{IS}$$

- y_t adalah *output gap* pada periode t. Jika $y_t > 0$, ekonomi sedang berjalan di atas kapasitas normalnya (*overheating*). Jika $y_t < 0$, ekonomi sedang lesu di bawah potensinya.
- $r_{t-1} - \pi_{t-1}$ adalah suku bunga riil efektif, yaitu BI-Rate dikurangi inflasi. Inilah biaya pinjaman "sesungguhnya" yang dirasakan pelaku ekonomi. Semakin tinggi suku bunga riil, semakin mahal biaya kredit, sehingga investasi dan konsumsi turun, yang tercermin dari tanda negatif pada koefisien β.
- r^* adalah suku bunga riil natural, yaitu tingkat suku bunga yang membuat ekonomi berada pada keseimbangan tanpa tekanan inflasi maupun resesi. Dalam spesifikasi ini r^* disetarakan dengan target inflasi $\pi^* = 2,5\%$.
- ρ_y mengukur seberapa persisten *output gap*. Nilai $\rho_y = 0,60$ berarti 60% kondisi *output* periode lalu terbawa ke periode sekarang (efek inersia).
- β mengukur sensitivitas *output* terhadap perubahan suku bunga riil. Nilai $\beta = 0,30$ berarti kenaikan suku bunga riil sebesar satu poin persentase akan menurunkan *output gap* sebesar 0,30 poin persentase.
- ϵ_t^{IS} adalah guncangan acak pada sisi permintaan, misalnya perubahan mendadak pada konsumsi rumah tangga atau investasi swasta yang tidak dapat diprediksi oleh model.

1.3.2 Hybrid New Keynesian Phillips Curve (Sisi Penawaran dan Inflasi)

Persamaan Kurva Phillips menghubungkan inflasi dengan *output gap* dan depresiasi nilai tukar. Versi *hybrid* yang digunakan di sini mencakup komponen *backward-looking* karena pelaku ekonomi di Indonesia cukup signifikan dalam membentuk ekspektasi inflasi berdasarkan inflasi periode lalu.

$$\pi_t = \rho_\pi \, \pi_{t-1} + \kappa \, y_t + \phi \, \Delta e_t + \epsilon_t^{PC}$$

- π_t adalah inflasi pada periode t, dinyatakan dalam poin persentase.
- ρ_π menangkap persistensi inflasi. Nilai $\rho_\pi = 0,85$ berarti inflasi cenderung "lengket" dan tidak berubah drastis antar periode, sebuah karakteristik empiris perekonomian Indonesia yang memiliki komponen ekspektasi adaptif cukup kuat.
- κ adalah kemiringan (*slope*) Kurva Phillips, yang mengukur seberapa besar *output gap* mendorong inflasi. Ketika ekonomi *overheating* ($y_t > 0$), permintaan melebihi kapasitas produksi sehingga harga-harga naik. Nilai $\kappa = 0,11$ diestimasi dari data historis Indonesia oleh Juhro [3].
- Δe_t adalah persen depresiasi nilai tukar rupiah terhadap USD. Tanda positif berarti rupiah melemah.
- ϕ adalah koefisien *exchange rate pass-through*, yang mengukur seberapa besar depresiasi rupiah diteruskan ke harga barang dan jasa domestik. Nilai $\phi = 0,30$ berarti depresiasi 10% akan menaikkan inflasi sekitar 3,0 poin persentase melalui kenaikan harga barang impor.
- ϵ_t^{PC} adalah guncangan acak pada sisi penawaran, misalnya lonjakan harga pangan akibat gagal panen atau kenaikan harga energi global.

1.3.3 Uncovered Interest Rate Parity (Nilai Tukar)

Persamaan UIP menjelaskan pergerakan nilai tukar berdasarkan selisih suku bunga domestik dan suku bunga luar negeri. Logika dasarnya sederhana. Jika BI-Rate lebih tinggi dari Fed Funds Rate, investor global akan memindahkan dananya ke Indonesia untuk memperoleh imbal hasil lebih tinggi, sehingga permintaan rupiah naik dan rupiah menguat.

$$\Delta e_t = -\left(r_t - r_t^{US}\right) + \rho_e + \epsilon_t^{FX}$$

- r_t^{US} adalah suku bunga acuan Amerika Serikat (Fed Funds Rate) pada kuartal ke-t. Tidak seperti parameter lain yang bernilai konstan, r_t^{US} mengikuti lintasan skenario yang merepresentasikan siklus pelonggaran yang berlanjut kemudian berbalik mengetat (*easing-to-hiking cycle*) sepanjang dua tahun ke depan. Lintasan yang ditetapkan adalah $r_t^{US} = (3{,}375\%,\ 3{,}125\%,\ 3{,}125\%,\ 3{,}125\%,\ 3{,}375\%,\ 3{,}625\%,\ 3{,}875\%,\ 4{,}125\%)$, yang menggambarkan kelanjutan pemotongan suku bunga pada empat kuartal pertama hingga menyentuh dasar 3,125%, ditahan selama dua kuartal, lalu dinaikkan kembali pada dua kuartal terakhir seiring pemulihan ekonomi.
- Tanda negatif pada selisih $r_t - r_t^{US}$ mencerminkan hubungan terbalik. Semakin besar selisih positif (BI-Rate lebih tinggi), semakin besar aliran modal masuk sehingga rupiah menguat (Δe_t negatif, atau apresiasi).
- ρ_e adalah premi risiko dasar nilai tukar, ditetapkan netral sebesar 0.
- ϵ_t^{FX} adalah guncangan acak pada pasar valuta asing, misalnya gejolak sentimen global atau perubahan persepsi risiko terhadap negara berkembang.
- Δe_t dinyatakan dalam persen depresiasi. Nilai tukar level e_t (dalam Rp/USD) diperbarui dengan rumus $e_t = e_{t-1} \cdot (1 + \Delta e_t/100)$.

1.3.4 Tiga Persamaan Proksi

Dari ketiga persamaan inti di atas, diturunkan tiga variabel proksi yang dibutuhkan untuk mengevaluasi *constraint* dan fungsi objektif.

Daya Beli Masyarakat. Proksi daya beli didefinisikan sebagai selisih antara pertumbuhan ekonomi dan inflasi. Jika *output* tumbuh lebih cepat dari inflasi, daya beli riil masyarakat meningkat. Konsep ini sederhana namun cukup untuk menangkap efek distributif kebijakan moneter untuk *scope* tugas besar ini.

$$PP_t = y_t - \pi_t$$

Neraca Transaksi Berjalan. Neraca transaksi berjalan (*current account*) adalah catatan arus barang, jasa, dan transfer antara Indonesia dengan negara lain. Persamaan ini menangkap dua kekuatan yang berlawanan. Depresiasi rupiah ($\Delta e_t > 0$) membuat ekspor lebih murah bagi pembeli asing sehingga memperbaiki neraca transaksi berjalan, tercermin dari tanda positif pada α_1. Sementara itu, *output* domestik yang tinggi ($y_t > 0$) meningkatkan impor karena permintaan dalam negeri membesar, tercermin dari tanda negatif pada α_2.

$$CA_t = \alpha_1\, \Delta e_t - \alpha_2\, y_t + \epsilon_t^{CA}$$

Sebagai catatan, nilai CA_t yang dihasilkan sudah dalam satuan persen terhadap PDB.

Imbal Hasil Surat Berharga Negara. Obligasi pemerintah Indonesia (SBN) menawarkan imbal hasil yang umumnya berada di atas BI-Rate karena investor menuntut kompensasi atas risiko fiskal. *Spread* antara imbal hasil SBN dan BI-Rate tidak boleh terlalu lebar karena menandakan kekhawatiran pasar terhadap kemampuan fiskal pemerintah.

$$y_t^{SBN} = r_t + \rho_{fiskal} + \epsilon_t^{SBN}$$

dengan ρ_{fiskal} adalah premi risiko fiskal dasar sebesar 1,50 poin persentase.

1.3.5 Guncangan Stokastik $\epsilon_t^{(\cdot)}$

Seluruh notasi $\epsilon_t^{(\cdot)}$ pada persamaan di atas adalah variabel acak yang disimulasikan dengan distribusi normal. Guncangan ini merepresentasikan kejadian tak terduga seperti lonjakan harga minyak dunia, bencana alam yang mengganggu rantai pasok, atau perubahan sentimen investor global. Kehadiran guncangan stokastik memiliki implikasi penting bagi proses *local search*. Evaluasi $J(s)$ untuk state yang sama bisa menghasilkan nilai sedikit berbeda karena realisasi guncangan yang berbeda. Untuk menjamin reproduktibilitas, seluruh peserta anda menggunakan *seed* tetap (`seed = 42`), sehingga guncangan yang dialami oleh tiap kandidat state identik antar satu sama lain.

Bab 2

Spesifikasi Masalah

2.1 Representasi State

Pada permasalahan ini, state yang digunakan untuk mewakili kebijakan suku bunga BI akan direpresentasikan dalam bentuk vektor suku bunga acuan BI-Rate untuk seluruh horizon kebijakan sekaligus yang dinyatakan oleh

$$s = (r_1, r_2, \ldots, r_T).$$

Tiap elemen r_t adalah BI-Rate yang berlaku pada kuartal ke-t, dengan $T = 8$ periode (2 tahun). Domain nilai r_t didiskritisasi dalam kelipatan 25 basis poin (0,25%), sesuai dengan konvensi dalam RDG BI yang umumnya mengubah suku bunga dalam kelipatan tersebut. Secara matematis, haruslah nilai r_t memenuhi

$$r_t \in \{3{,}00\%,\ 3{,}25\%,\ 3{,}50\%,\ \ldots,\ 8{,}00\%\}$$

Ukuran ruang state dari formulasi ini adalah $21^8 \approx 3,78 \times 10^{10}$ kemungkinan (21 tingkat suku bunga diskrit, 8 periode). Ukuran ini, tentunya akan sangat tidak efisien jika ditelusuri jika menggunakan algoritma *brute-force*. Dengan demikian, diperlukan algoritma *local search* untuk mengeksplorasi ruang pencarian secara lebih strategis.

2.2 Aturan dan Batasan Permasalahan

Permasalahan ini dibatasi oleh lima *constraint* yang merepresentasikan kondisi neraca keuangan riil Indonesia, mencakup aspek domestik maupun eksternal. *Constraint* C1 bersifat struktural terhadap representasi state itu sendiri sehingga dapat diverifikasi langsung dari vektor s tanpa perlu simulasi. *Constraint* C2 sampai C5 melibatkan variabel makroekonomi hasil simulasi sehingga pelanggarannya baru dapat dideteksi setelah fungsi `compute_objective(s)` dijalankan.

C1. Batas dan Diskritisasi Suku Bunga.

$$r_t \in [r_{min}, r_{max}] = [3{,}00\%,\ 8{,}00\%], \quad r_t \bmod 0{,}25\% = 0 \qquad \forall t$$

Constraint ini memastikan setiap r_t berada pada rentang suku bunga yang realistis untuk ekonomi berkembang dan selalu dalam kelipatan 25 basis poin sesuai konvensi perubahan suku bunga BI.

C2. Batas Defisit Transaksi Berjalan.

$$CA_t(s) \geq -\theta, \qquad \theta = 3{,}00\%$$

Defisit transaksi berjalan tidak boleh melebihi ambang 3,00% PDB, merujuk pada batas aman IMF untuk negara *emerging market*. *Constraint* ini merepresentasikan kondisi terkini Indonesia yang baru mengalami

defisit transaksi berjalan pertama setelah 72 bulan surplus berturut-turut, sehingga ruang gerak kebijakan yang memperlebar defisit lebih jauh perlu dibatasi.

C3. Batas *Spread* Imbal Hasil Obligasi Pemerintah.

$$y_t^{SBN} - r_t \leq \sigma_{max}, \qquad \sigma_{max} = 2{,}50\%$$

Selisih antara imbal hasil SBN dan BI-Rate tidak boleh melebar melebihi $\sigma_{max} = 2{,}50$ poin persentase, untuk menghindari risiko *fiscal dominance*, yaitu situasi di mana biaya pembiayaan utang pemerintah membengkak akibat sentimen pasar terhadap arah kebijakan moneter. Risiko ini secara eksplisit menjadi perhatian dalam diskusi perluasan kewenangan BI membeli SBN di pasar perdana pada UU P2SK.

C4. Konvergensi Target Inflasi di Akhir Horizon.

$$|\pi_T - \pi^*| \leq 1\%, \qquad \pi^* = 2{,}5\%$$

Lintasan kebijakan harus membawa inflasi kembali ke dalam rentang sasaran resmi $2{,}5\% \pm 1\%$ pada akhir horizon $T = 8$. *Constraint* ini menjamin bahwa state yang dihasilkan bukan sekadar optimal secara matematis, tetapi juga kredibel sebagai kebijakan publik sesuai kesepakatan sasaran inflasi Pemerintah-BI untuk periode 2026-2027.

C5. Gradualisme Kebijakan (*Penalized Interest Rate Smoothing*).

$$|r_t - r_{t-1}| \leq 50 \text{ bps} \qquad \forall t$$

Berbeda dengan C1-C4 yang bersifat *hard constraint* (state harus memenuhi agar dinyatakan *feasible*), C5 diperlakukan sebagai *penalized constraint*, dimana perubahan suku bunga melebihi 50 bps diizinkan, namun dikenakan penalti kuadratik pada fungsi objektif. Pendekatan ini mencerminkan realitas kebijakan moneter, yaitu perubahan suku bunga yang terlalu agresif dapat mengguncang ekspektasi pasar dan merusak kredibilitas *forward guidance* [4], namun dalam kondisi luar biasa (seperti krisis) bank sentral tetap dapat mengambil langkah drastis.

2.2.1 Definisi *Feasible* dan Peran π_T

Status *feasible*. Sebuah state s dinyatakan ***feasible*** jika dan hanya jika seluruh *hard constraint* C1, C2, C3, dan C4 terpenuhi. *Constraint* C5 (gradualisme) dikeluarkan dari definisi *feasible* karena bersifat *soft/penalized*, dimana perubahan suku bunga lebih dari 50 bps tidak membatalkan kelayakan kebijakan, melainkan hanya dikenakan penalti pada fungsi objektif $J_{penalized}(s)$. Secara formal,

$$\text{feasible}(s) \iff \text{C1} \wedge \text{C2} \wedge \text{C3} \wedge \text{C4}$$

Peran π_T (inflasi terminal). π_T adalah nilai inflasi pada akhir horizon kebijakan ($t = T = 8$). Nilai ini menjadi variabel kunci karena

- π_T digunakan untuk verifikasi *constraint* C4, yaitu $|\pi_T - \pi^*| \leq 1\%$. Constraint ini menjamin bahwa lintasan suku bunga yang dihasilkan algoritma tidak hanya optimal secara matematis, tetapi juga kredibel sebagai kebijakan publik. Inflasi harus kembali ke dalam rentang sasaran $2{,}5\% \pm 1\%$ pada akhir horizon.
- π_T dilaporkan bersama hasil akhir setiap algoritma agar pembaca dapat menilai secara langsung apakah kebijakan yang direkomendasikan berhasil mengarahkan inflasi menuju target.
- Bersama RMSD_π, π_T memberikan gambaran lengkap. RMSD_π akan mengukur deviasi rata-rata sepanjang seluruh horizon, sedangkan π_T menunjukkan kondisi akhir yang dicapai.

2.3 *Succesor Function*

Successor function didefinisikan sebagai cara membangkitkan state tetangga dari sebuah state s. Dalam kerangka *local search*, tetangga adalah state yang "dekat" dengan state saat ini menurut suatu metrik tertentu. Pada permasalahan ini, akan digunakan tiga jenis *move*.

- *Move 1. Perubahan Titik.* Operator ini mengubah satu elemen r_t sebesar ± 25 basis poin (satu tingkat diskritisasi) dan mempertahankan elemen lain tetap. Karena dinamika model bersifat kausal, perubahan pada r_t mempengaruhi seluruh *trajectory* dari periode t hingga T. Secara implementatif, setelah r_t dimodifikasi, simulasi ekonomi dijalankan ulang dari periode t sampai T. *Move* ini bersifat lokal dan presisi, cocok untuk *fine-tuning* di sekitar solusi yang sudah cukup baik.
- *Move 2. Pergeseran Window.* Operator ini memilih satu periode t dan menggeser seluruh sub-state dari r_t hingga r_T sebesar ± 25 basis poin. *Move* ini mensimulasikan perubahan arah kebijakan secara menyeluruh. Menggeser ke atas merepresentasikan pergeseran ke arah yang lebih *hawkish* (mengetatkan kebijakan), sedangkan menggeser ke bawah merepresentasikan pergeseran ke arah yang lebih *dovish* (melonggarkan kebijakan). Karena pergeseran bersifat seragam pada seluruh sisa horizon, *move* ini mempertahankan *constraint* C2 selama pergeseran tidak menyebabkan pelanggaran batas atas atau bawah.
- *Move 3. Crossover.* Operator ini khusus digunakan pada *Genetic Algorithm* dan bekerja pada dua state induk. Sebuah titik potong $k \in \{1, \ldots, T-1\}$ dipilih, lalu segmen $[r_1, \ldots, r_k]$ dari induk pertama ditukar dengan segmen yang sama dari induk kedua, menghasilkan dua state anak. *Move* ini memungkinkan pertukaran blok kebijakan antar kandidat, mempercepat eksplorasi dengan menggabungkan segmen yang baik dari dua state berbeda.

Untuk algoritma *Hill-Climbing* dan *Simulated Annealing*, tetangga dibangkitkan dengan memilih secara acak antara *Move 1* dan *Move 2* pada tiap iterasi. Untuk *Genetic Algorithm*, populasi terdiri dari sekumpulan state s, dengan mutasi menggunakan *Move 1* dan *crossover* menggunakan *Move 3*. Perlu dicatat bahwa seluruh *move* harus tetap menjaga *constraint* C1 setelah modifikasi. Jika hasil modifikasi melanggar C1, state tetangga diperbaiki terlebih dahulu (dengan *clipping* ke rentang yang diizinkan) sebelum dievaluasi.

2.4 *Objective Function*

2.4.1 Definisi *Objective Function*

Fungsi objektif $J(s)$ adalah ukuran kuantitatif yang menentukan seberapa baik sebuah state s dalam menyeimbangkan tiga tujuan kebijakan moneter Bank Indonesia, yaitu stabilitas harga, stabilitas nilai tukar, dan daya beli masyarakat. *Local search* bertugas mencari state yang memaksimalkan $J(s)$ pada ruang state $\mathcal{S}$ yang memenuhi seluruh *constraint* C1 sampai C5.

$$s^* = \arg\max_{s \in \mathcal{S}} J(s)$$

Nilai $J(s)$ dihitung melalui dua langkah. Pertama, state s disimulasikan pada model makroekonomi untuk menghasilkan *trajectory* variabel $\{y_t, \pi_t, e_t, PP_t, CA_t, y_t^{SBN}\}_{t=1}^{T}$. Kedua, *trajectory* tersebut dievaluasi terhadap target kebijakan dan *constraint* untuk menghasilkan vektor suku bunga akhir.

2.4.2 *Loss Function* Teoretik

Loss function $L(s)$ mengikuti kerangka *quadratic loss function* Svensson [5] yang menjadi standar analisis kebijakan bank sentral. Secara konseptual, $L(s)$ mengukur total "kerugian" atau deviasi variabel makroekonomi dari targetnya sepanjang horizon kebijakan. Setiap komponen dihitung sebagai kuadrat deviasi, sehingga deviasi besar dikenakan penalti yang jauh lebih berat daripada deviasi kecil (sifat kuadratik). Perumusan ini mencerminkan preferensi bank sentral yang lebih memilih deviasi kecil di banyak periode dibandingkan satu deviasi besar di satu periode.

$$L(s) = \sum_{t=1}^{T} \delta^t \Big[w_\pi (\pi_t - \pi^*)^2 + w_y \, y_t^2 + w_{pp} (PP_t - PP^*)^2 + w_r (r_t - r_{t-1})^2 \Big]$$

Keempat suku di dalam kurung siku memiliki interpretasi kebijakan yang spesifik. Suku $w_\pi(\pi_t - \pi^*)^2$ menghukum deviasi inflasi dari sasaran $\pi^* = 2{,}5\%$. Suku $w_y \, y_t^2$ menghukum *output gap* yang tidak nol pada kedua arah, karena ekonomi yang *overheating* ($y_t > 0$) maupun resesi ($y_t < 0$) sama-sama tidak diinginkan. Suku $w_{pp}(PP_t - PP^*)^2$ menghukum deviasi daya beli dari target $PP^* = 0$, yaitu kondisi ideal di mana pertumbuhan

riil tepat mengimbangi inflasi sehingga daya beli masyarakat tidak tergerus. Suku $w_r(r_t - r_{t-1})^2$ menghukum volatilitas suku bunga yang berlebihan, mendorong *interest rate smoothing* sebagaimana dikehendaki *constraint* C2 secara lunak pada fungsi objektif.

Faktor diskonto $\delta = 0{,}95$ memberi bobot lebih besar pada periode awal dibanding periode akhir. Dengan $\delta < 1$, deviasi pada kuartal pertama memiliki kontribusi lebih signifikan terhadap total *loss* dibanding deviasi yang sama pada kuartal kedelapan, mencerminkan preferensi bank sentral terhadap stabilitas jangka pendek dan ketidakpastian yang meningkat seiring bertambahnya horizon proyeksi.

Bobot w_π, w_y, w_{pp}, w_r menentukan prioritas relatif antar tujuan kebijakan. Konfigurasi default $(w_\pi, w_y, w_{pp}, w_r) = (1{,}0,\ 0{,}5,\ 0{,}3,\ 0{,}2)$ menempatkan stabilitas inflasi sebagai prioritas tertinggi, diikuti oleh *output gap*, daya beli, dan *smoothing* suku bunga sebagai prioritas terendah.

2.4.3 *Loss Function* Final

Loss function final yang digunakan dalam implementasi adalah $J_{penalized}(s)$, yaitu negatif dari $L(s)$ yang telah ditambah penalti *constraint*. Negasi dilakukan karena algoritma *local search* secara konvensional bekerja dengan memaksimalkan fungsi objektif, sedangkan $L(s)$ adalah fungsi yang ingin diminimalkan.

$$J_{penalized}(s) = -L(s) - \mu \sum_{i=2}^{5} \max\big(0,\ \text{violation}_i(s)\big)^2$$

Suku pertama, $-L(s)$, adalah negatif dari *quadratic loss* yang mengubah permasalahan minimisasi $L(s)$ menjadi maksimisasi. Seluruh suku pada $L(s)$ bernilai non-negatif karena berbentuk kuadrat, sehingga $J(s) = -L(s) \leq 0$ untuk semua s. Nilai $J(s)$ yang semakin mendekati nol menandakan *trajectory* ekonomi yang semakin dekat ke target. Nilai maksimum teoretis $J(s) = 0$ hanya tercapai jika semua deviasi sama dengan nol di seluruh periode, kondisi yang secara praktis mustahil karena kehadiran guncangan stokastik ϵ_t.

Suku kedua adalah penalti untuk *constraint* C2 sampai C5 yang dilanggar. Pendekatan *penalty method* dipilih karena *constraint* C2 sampai C5 melibatkan variabel hasil simulasi sehingga tidak dapat diverifikasi sebelum simulasi dijalankan, berbeda dengan C1 yang dapat dicek langsung dari vektor s. Bobot penalti μ ditetapkan sebesar 100, cukup besar untuk mendominasi perbaikan pada $L(s)$ sehingga algoritma terdorong kuat menuju region *feasible* sebelum melakukan *fine-tuning* terhadap nilai $J(s)$. State yang memenuhi seluruh *constraint* memiliki suku penalti nol sehingga $J_{penalized}(s) = -L(s)$. Setiap *constraint* $i \in \{2, 3, 4, 5\}$ memiliki pelanggaran $\text{violation}_i(s)$ yang dihitung sebagai berikut.

$$\begin{aligned}
\text{violation}_2(s) &= \max_t\big(0,\ -\theta - CA_t\big) \\
\text{violation}_3(s) &= \max_t\big(0,\ (y_t^{SBN} - r_t) - \sigma_{max}\big) \\
\text{violation}_4(s) &= \max\big(0,\ |\pi_T - \pi^*| - 1\%\big) \\
\text{violation}_5(s) &= \max_t\big(0,\ |r_t - r_{t-1}| - 0{,}50\%\big)
\end{aligned}$$

Pelanggaran dikuadratkan agar pelanggaran besar dikenakan penalti yang jauh lebih berat daripada pelanggaran kecil, mendorong algoritma untuk memprioritaskan perbaikan pada *constraint* yang paling parah dilanggar.

2.5 Formulasi *Initial State*

Initial state s_0 dibentuk secara acak dari seluruh rentang yang diizinkan. Setiap r_t dipilih secara acak dari distribusi uniform pada rentang $[r_{min}, r_{max}]$ lalu dibulatkan ke kelipatan 0,25% terdekat, sehingga *constraint* C1 secara implisit terpenuhi. Jika hasil pembangkitan acak tidak memenuhi C2 sampai C4, state tetap diterima sebagai initial state dalam status *infeasible*, dan proses *local search* bertugas mengarahkan pencarian menuju state yang *feasible* sekaligus optimal melalui mekanisme penalti pada fungsi objektif.

Kondisi awal sistem (periode $t = 0$).

$r_0 = 5{,}75\%$	BI-Rate
$\pi_0 = 3{,}34\%$	Inflasi tahunan Juni 2026
$y_0 = 0{,}00\%$	*Output gap* awal
$e_0 = 17.885$ Rp/USD	Nilai Dollar ke Rupiah
$r_t^{US} = (3{,}375\%,\ 3{,}125\%,\ 3{,}125\%,\ 3{,}125\%,\ 3{,}375\%,\ 3{,}625\%,\ 3{,}875\%,\ 4{,}125\%)$	Fed Rate

Parameter constraint dan target.

$T = 8$	Horizon kebijakan (kuartalan, 2 tahun)
$r_{min} = 3{,}00\%$	Batas bawah suku bunga
$r_{max} = 8{,}00\%$	Batas atas suku bunga
$\pi^* = 2{,}5\%$	Sasaran inflasi tengah
$\Delta_{min} = 1{,}50\%$	Selisih minimum BI-Rate terhadap Fed Rate
$\theta = 3{,}00\%$	Batas defisit transaksi berjalan (% PDB)
$\sigma_{max} = 2{,}50\%$	*Spread* maksimum SBN terhadap BI-Rate
$\delta = 0{,}95$	Faktor diskonto *loss function*

Bobot komponen fungsi objektif.

$w_\pi = 1{,}0$	Bobot inflasi
$w_y = 0{,}5$	Bobot *output gap*
$w_{pp} = 0{,}3$	Bobot daya beli
$w_r = 0{,}2$	Bobot *interest rate smoothing*

2.6 *Model Evaluation Metric*

Kualitas solusi akhir tidak dapat diukur dari $J_{penalized}(s)$ saja, karena nilai $J(s)$ dipengaruhi oleh bobot w dan δ yang meskipun bersifat tetap, merepresentasikan satu konfigurasi preferensi spesifik. Diperlukan metrik evaluasi yang berdiri di luar fungsi objektif, bersifat netral, dan langsung mengukur seberapa dekat *trajectory* inflasi terhadap target kebijakan. Metrik tersebut adalah ***Root Mean Square Deviation* (RMSD)**.

Sebelum membahas RMSD, perlu diklarifikasi pemisahan antara komponen yang bersifat tetap (*fixed*) dan komponen yang bebas diatur (*free*) dalam eksperimen.

- *Komponen tetap.* Seluruh parameter yang tertuang dalam Formulasi Initial State pada spesifikasi bersifat wajib dan tidak boleh diubah. Ini mencakup parameter struktural model ($\kappa, \beta, \phi, \rho_y, \rho_\pi, \rho_e, \alpha_1, \alpha_2, \rho_{fiskal}$), kondisi awal ($r_0, \pi_0, y_0, e_0, r_0^{US}$), target ($\pi^*, PP^*$), parameter *constraint* ($r_{min}, r_{max}, \Delta_{min}, \theta, \sigma_{max}$), bobot fungsi objektif (w_π, w_y, w_{pp}, w_r), faktor diskonto (δ), horizon (T), dan protokol *shock* (`seed = 42`). Lingkungan simulasi yang identik menjamin perbedaan nilai RMSD semata-mata mencerminkan perbedaan kualitas algoritma, bukan perbedaan "dunia" yang disimulasikan.
- *Komponen bebas.* Hal-hal yang dapat divariasikan hanyalah aspek yang berkaitan dengan algoritma *local search* itu sendiri, yaitu pilihan algoritma (Hill-Climbing, Simulated Annealing, atau Genetic Algorithm), jumlah iterasi (`max_iter`), jumlah generasi dan ukuran populasi untuk GA, suhu awal dan laju pendinginan untuk SA, probabilitas mutasi, strategi seleksi, serta *random seed* untuk inisialisasi state dan pemilihan *neighbor*. Variasi pada komponen bebas inilah yang menghasilkan nilai $J(s)$ dan RMSD yang berbeda antar eksperimen meskipun ekonomi yang disimulasikan identik.

RMSD_π mengukur akar rata-rata kuadrat deviasi inflasi dari target $\pi^* = 2{,}5\%$ sepanjang horizon $T = 8$. Metrik ini tidak melibatkan bobot, diskonto, maupun penalti *constraint*, sehingga murni mengukur seberapa dekat *trajectory* inflasi terhadap target.

$$\text{RMSD}_\pi = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(\pi_t - \pi^*)^2}$$

Secara implementatif, RMSD_π dihitung setelah `simulate_economy` menghasilkan *trajectory*.

```
rmsd_pi = np.sqrt(np.mean((traj["pi"] - pi_star)**2))
```

Nilai RMSD_π bersifat non-negatif, dengan nol sebagai nilai terbaik (inflasi tepat di target sepanjang seluruh horizon). Semakin kecil RMSD_π, semakin baik state suku bunga yang ditemukan. Berdasarkan karakteristik numerik dari parameter dan *shock* yang ditetapkan, interpretasi kualitatif nilai RMSD_π adalah sebagai berikut.

- $\text{RMSD}_\pi < 0{,}50$ dinilai sangat baik. Inflasi rata-rata hanya menyimpang kurang dari setengah poin persentase dari target, menunjukkan algoritma *local search* berhasil menemukan state yang efektif menavigasi *trade-off* kebijakan.
- $0{,}50 \leq \text{RMSD}_\pi < 1{,}00$ dinilai baik dan masih dalam rentang toleransi sasaran inflasi $\pm 1\%$.
- $\text{RMSD}_\pi \geq 1{,}00$ dinilai buruk karena rata-rata deviasi inflasi melampaui batas toleransi resmi, mengindikasikan algoritma terjebak pada solusi yang tidak memenuhi *constraint* C4 atau gagal mengeksplorasi ruang state secara memadai.

Selain RMSD_π, dapat pula dihitung RMSD_y terhadap target $y^* = 0$ dan RMSD_{PP} terhadap target $PP^* = 0$ untuk analisis lebih lanjut. Namun dalam konteks perbandingan utama, hanya RMSD_π yang digunakan.

2.7 Daftar Command Utama

Berikut adalah daftar fungsi yang perlu diimplementasikan dalam program, dikelompokkan berdasarkan tiga fase utama dalam pipeline eksekusi. Setiap command ditulis dalam format `nama_fungsi(param1, param2, ...)` diikuti oleh tujuan dan daftar parameternya.

Fase 1. Persiapan dan Simulasi Ekonomi

`generate_shocks(T=8, seed=42)`

Menghasilkan guncangan stokastik untuk seluruh horizon simulasi. Seluruh peserta wajib menggunakan *seed* yang sama agar hasil dapat dibandingkan.

- `T` (int) jumlah periode horizon kebijakan.
- `seed` (int) *seed* untuk reproduktibilitas, ditetapkan 42.
- *Return* `dict` berisi lima array `NumPy` berdimensi `(T,)` untuk guncangan IS, PC, FX, CA, dan SBN.

`simulate_economy(s, shocks, params)`

Menjalankan simulasi model makroekonomi secara rekursif dari $t = 1$ sampai T. Fungsi ini adalah inti komputasi program karena dipanggil setiap kali sebuah state dievaluasi.

- `s` (list[float]) state suku bunga $s = (r_1, \ldots, r_T)$, tiap elemen dalam persen.
- `shocks` (dict) guncangan stokastik hasil `generate_shocks`.
- `params` (dict) parameter model $(\kappa, \beta, \phi, \rho_y, \rho_\pi, \rho_e, \alpha_1, \alpha_2, \rho_{fiskal})$ dan kondisi awal $(\pi_0, y_0, e_0, r_0^{US})$.
- *Return* `dict` berisi enam array berdimensi `(T,)` untuk *trajectory* variabel $\{y_t, \pi_t, e_t, PP_t, CA_t, y_t^{SBN}\}_{t=1}^{T}$.

Fase 2. Evaluasi State

`compute_purchasing_power(y, pi)`

Menghitung proksi daya beli masyarakat sebagai selisih *output gap* dan inflasi.

- `y` (array[float]) *output gap* tiap periode.
- `pi` (array[float]) inflasi tiap periode.
- *Return* `array[float]` berisi $PP_t = y_t - \pi_t$ untuk $t = 1, \ldots, T$.

`compute_current_account(de, y, eps_ca)`

Menghitung proksi neraca transaksi berjalan berdasarkan depresiasi nilai tukar dan *output gap*.

- `de` (array[float]) persen depresiasi Δe_t tiap periode.
- `y` (array[float]) *output gap* tiap periode.
- `eps_ca` (array[float]) guncangan stokastik ϵ_t^{CA}.
- *Return* `array[float]` berisi $CA_t = \alpha_1 \Delta e_t - \alpha_2 y_t + \epsilon_t^{CA}$, dalam satuan persen PDB.

`compute_sbn_yield(r, eps_sbn, rho_fiskal)`

Menghitung proksi imbal hasil Surat Berharga Negara.

- `r` (array[float]) BI-Rate tiap periode.
- `eps_sbn` (array[float]) guncangan stokastik ϵ_t^{SBN}.
- `rho_fiskal` (float) premi risiko fiskal, ditetapkan 1,50.
- *Return* `array[float]` berisi $y_t^{SBN} = r_t + \rho_{fiskal} + \epsilon_t^{SBN}$.

`check_constraints(s, trajectory, params)`

Memvalidasi seluruh *constraint* C1 sampai C5 terhadap sebuah state dan *trajectory* simulasinya, serta menghitung besar pelanggaran untuk penalti fungsi objektif. Status *feasible* hanya mempertimbangkan *hard constraint* C1 sampai C4; C5 adalah *soft constraint* yang tidak mempengaruhi status *feasible*.

- `s` (list[float]) state suku bunga.
- `trajectory` (dict) hasil simulasi dari `simulate_economy`.
- `params` (dict) parameter *constraint* $(r_{min}, r_{max}, r_0, \theta, \sigma_{max}, \pi^*)$.
- *Return* `dict` berisi `feasible` (bool) dan `violations` (dict) dengan kunci `C2, C3, C4, C5` yang bernilai besar pelanggaran masing-masing. C1 dapat diverifikasi langsung dari `s` tanpa `trajectory`.

`compute_objective(s, shocks, params, mu=100)`

Fungsi utama yang menghitung nilai $J_{penalized}(s)$ untuk sebuah state. Fungsi ini memanggil `simulate_economy`, seluruh fungsi proksi, dan `check_constraints` secara berurutan.

- `s` (list[float]) state suku bunga.
- `shocks` (dict) guncangan stokastik.
- `params` (dict) seluruh parameter model, *constraint*, dan bobot $(w_\pi, w_y, w_{pp}, w_r, \delta$, serta parameter yang dibutuhkan `simulate_economy` dan `check_constraints`).
- `mu` (float) bobot penalti *constraint*, default 100.
- *Return* `float` nilai $J_{penalized}(s) = -L(s) - \mu \sum_{i=2}^{5} \text{violation}_i(s)^2$.

`compute_rmsd(trajectory, pi_star=2.5)`

Menghitung RMSD_π dari *trajectory* hasil simulasi. Fungsi ini murni menghitung metrik evaluasi tanpa melibatkan bobot subjektif atau penalti *constraint*.

- `trajectory` (dict) hasil simulasi dari `simulate_economy`, minimal mengandung kunci `"pi"`.
- `pi_star` (float) target inflasi, default 2,5.
- *Return* `float` nilai $\text{RMSD}_\pi = \sqrt{\frac{1}{T}\sum(\pi_t - \pi^*)^2}$.

Fase 3. Pencarian *Local Search*

`generate_initial_state(T=8, r_min=3.0, r_max=8.0, rng=None)`

Membentuk initial state s_0 secara acak dengan memilih setiap r_t independen dari distribusi uniform pada $[r_{min}, r_{max}]$ lalu dibulatkan ke kelipatan 0,25%, sehingga C1 terpenuhi secara implisit.

- `T` (int) horizon kebijakan.
- `r_min` (float) batas bawah suku bunga.
- `r_max` (float) batas atas suku bunga.
- `rng` (numpy.random.Generator) generator bilangan acak.
- *Return* `list[float]` state s_0 dengan panjang T.

`generate_neighbor(s, move_type=None, rng=None)`

Menghasilkan satu state tetangga dari s menggunakan *Move 1* atau *Move 2* yang dipilih secara acak. Hasil modifikasi diperbaiki terhadap C1 sebelum dikembalikan.

- `s` (list[float]) state asal.
- `move_type` (int or None) jenis *move*. Jika `None`, dipilih acak antara 1 dan 2.
- `rng` (numpy.random.Generator) generator bilangan acak.
- *Return* `list[float]` state tetangga s'.

`generate_neighbor_crossover(s1, s2, rng=None)`

Menghasilkan dua state anak dari dua state induk melalui *crossover* satu titik potong, khusus untuk *Genetic Algorithm*.

- `s1` (list[float]) state induk pertama.
- `s2` (list[float]) state induk kedua.
- `rng` (numpy.random.Generator) generator bilangan acak.
- *Return* `tuple` dua `list[float]` sebagai state anak.

`hill_climbing(s0, max_iter, shocks, params)`

Algoritma *Hill-Climbing* dasar. Mulai dari s_0, pada tiap iterasi membangkitkan satu tetangga. Jika $J(s') > J(s_{current})$, pindah ke s'. Jika tidak, tetap di $s_{current}$. Berhenti setelah `max_iter` atau tidak ada perbaikan dalam k iterasi berturut-turut.

- `s0` (list[float]) initial state.
- `max_iter` (int) jumlah maksimum iterasi.
- `shocks` (dict) guncangan stokastik.
- `params` (dict) parameter model dan bobot.
- *Return* `dict` berisi `best_state` (list[float]), `best_score` (float), dan `history` (list[float]) nilai $J(s)$ per iterasi.

`simulated_annealing(s0, max_iter, T0, cooling_rate, shocks, params)`

Algoritma *Simulated Annealing*. Mirip *Hill-Climbing* tetapi menerima tetangga lebih buruk dengan probabilitas $\exp((J(s') - J(s))/T_{temp})$. Suhu T_{temp} menurun secara geometrik tiap iterasi dengan faktor `cooling_rate`.

- `s0` (list[float]) initial state.
- `max_iter` (int) jumlah maksimum iterasi.
- `T0` (float) suhu awal.
- `cooling_rate` (float) faktor penurunan suhu per iterasi, $\in (0, 1)$.
- `shocks` (dict) guncangan stokastik.
- `params` (dict) parameter model dan bobot.
- *Return* sama dengan `hill_climbing`.

`genetic_algorithm(pop_size, generations, mutation_rate, shocks, params)`

Algoritma *Genetic Algorithm*. Diawali dengan `pop_size` individu acak. Tiap generasi melakukan seleksi turnamen, *crossover* satu titik, dan mutasi dengan probabilitas `mutation_rate`. Elitisme menjaga individu terbaik tetap hidup ke generasi berikutnya.

- `pop_size` (int) jumlah individu dalam populasi.
- `generations` (int) jumlah generasi.
- `mutation_rate` (float) probabilitas mutasi per individu, $\in (0, 1)$.
- `shocks` (dict) guncangan stokastik.
- `params` (dict) parameter model dan bobot.
- *Return* `dict` berisi `best_state` (list[float]), `best_score` (float), dan `history` (list[float]) nilai $J(s)$ terbaik per generasi.

Fungsi Utilitas Tambahan

`round_to_bps(value)`

Membulatkan nilai suku bunga ke kelipatan 25 basis poin terdekat.

- `value` (float) nilai suku bunga dalam persen.
- *Return* `float` nilai yang telah dibulatkan.

`clip_state(s, r_min=3.0, r_max=8.0)`

Menjepit seluruh elemen state ke rentang $[r_{min}, r_{max}]$ dan membulatkannya. Berguna untuk memperbaiki state setelah modifikasi oleh *move*.

- `s` (list[float]) state yang akan diperbaiki.
- `r_min` (float) batas bawah.
- `r_max` (float) batas atas.
- *Return* `list[float]` state yang telah memenuhi C1.

2.8 Contoh Input dan Output

Contoh berikut menggunakan seluruh parameter standar dari Bagian 14 spesifikasi. Kode ditulis dalam Python dengan dependensi `numpy` saja.

Contoh 1. Inisialisasi Parameter dan *Shock*

Input

```
import numpy as np

T = 8
params = {
    "r0": 5.75, "pi0": 3.34, "y0": 0.00, "e0": 17885,
    "r0_US": 3.625, "pi_star": 2.5, "pp_star": 0.0,
    "rho_y": 0.60, "beta": 0.30, "kappa": 0.11, "phi": 0.30,
    "rho_pi": 0.85, "rho_e": 0.00, "r_star": 2.5,
    "alpha1": 0.20, "alpha2": 0.15, "rho_fiskal": 1.50,
    "r_min": 3.00, "r_max": 8.00,
    "delta_min": 1.50, "theta": 3.00, "sigma_max": 2.50,
    "delta": 0.95, "mu": 100,
    "w_pi": 1.0, "w_y": 0.5, "w_pp": 0.3, "w_r": 0.2,
}

shocks = generate_shocks(T=T, seed=42)
```

Contoh 2. Pembangkitan Initial State

Pemanggilan `generate_initial_state` menghasilkan state acak yang memenuhi C1 dan C2.

Input

```
rng = np.random.default_rng(42)
s0 = generate_initial_state(T=8, r_min=3.0, r_max=8.0, rng=rng)
```

Output

```
>>> s0
[6.00, 6.25, 6.25, 6.00, 5.75, 5.50, 5.25, 5.25]
```

Contoh 3. Simulasi Ekonomi

State s_0 disimulasikan pada model makroekonomi untuk memperoleh *trajectory* variabel.

Input

```
traj = simulate_economy(s0, shocks, params)
```

Output

```
>>> for t in range(T):
...     print(f"t={t+1}: r={s0[t]:.2f}%  y={traj['y'][t]:+.2f}  "
...           f"pi={traj['pi'][t]:.2f}%  e={traj['e'][t]:.0f}  "
...           f"PP={traj['PP'][t]:+.2f}  CA={traj['CA'][t]:+.2f}%")

t=1: r=6.00%  y=-0.12  pi=3.58%  e=17742  PP=-3.70  CA=-1.18%
t=2: r=6.25%  y=-0.35  pi=3.13%  e=17605  PP=-3.48  CA=-0.87%
t=3: r=6.25%  y=-0.55  pi=2.97%  e=17480  PP=-3.52  CA=-0.61%
t=4: r=6.00%  y=-0.48  pi=2.81%  e=17368  PP=-3.28  CA=-0.43%
t=5: r=5.75%  y=-0.30  pi=2.72%  e=17287  PP=-3.02  CA=-0.27%
t=6: r=5.50%  y=-0.12  pi=2.65%  e=17231  PP=-2.77  CA=-0.13%
t=7: r=5.25%  y=+0.04  pi=2.59%  e=17200  PP=-2.55  CA=-0.02%
t=8: r=5.25%  y=+0.13  pi=2.55%  e=17185  PP=-2.43  CA=+0.07%
```

Contoh 4. Evaluasi Objective dan *Constraint*

Input

```
result = check_constraints(s0, traj, params)
score = compute_objective(s0, shocks, params)
rmsd = compute_rmsd(traj, pi_star=2.5)
```

Output

```
>>> result["feasible"]
True

>>> result["violations"]
{'C2': 0.0, 'C3': 0.0, 'C4': 0.0}

>>> score
-8.935

>>> rmsd
0.476
```

Contoh 5. Menjalankan Ketiga Algoritma

Input

```
rng = np.random.default_rng(42)
s0 = generate_initial_state(T=8, r_min=3.0, r_max=8.0, rng=rng)

result_hc = hill_climbing(s0, max_iter=1000, shocks=shocks,
                          params=params)

rng = np.random.default_rng(42)
sa_s0 = generate_initial_state(T=8, r_min=3.0, r_max=8.0, rng=rng)
result_sa = simulated_annealing(sa_s0, max_iter=1000, T0=10.0,
                                cooling_rate=0.995, shocks=shocks,
                                params=params)

result_ga = genetic_algorithm(pop_size=50, generations=200,
                              mutation_rate=0.15, shocks=shocks,
                              params=params)

rmsd_hc = compute_rmsd(simulate_economy(result_hc["best_state"],
                       shocks, params))
rmsd_sa = compute_rmsd(simulate_economy(result_sa["best_state"],
                       shocks, params))
rmsd_ga = compute_rmsd(simulate_economy(result_ga["best_state"],
                       shocks, params))
```

Output

```
>>> print(f"HC␣␣:␣J={result_hc['best_score']:.4f}␣␣"
...       f"state={result_hc['best_state']}")
HC  : J=-8.9352  state=[6.00, 6.25, 6.25, 6.00, 5.75, 5.50, 5.25, 5.25]

>>> print(f"SA␣␣:␣J={result_sa['best_score']:.4f}␣␣"
...       f"state={result_sa['best_state']}")
SA  : J=-5.1823  state=[5.75, 5.50, 5.25, 5.00, 5.00, 5.00, 4.75, 4.75]

>>> print(f"GA␣␣:␣J={result_ga['best_score']:.4f}␣␣"
...       f"state={result_ga['best_state']}")
GA  : J=-4.9571  state=[5.75, 5.50, 5.25, 5.00, 5.00, 4.75, 4.75, 4.75]
```

Contoh 6. Konvergensi *Loss* per Iterasi

Output (nilai $J(s)$ pada beberapa iterasi terpilih)

```
=== Hill-Climbing Convergence (max_iter=1000) ===
Iter    10: J(s) = -9.2431
Iter    50: J(s) = -9.0124
Iter   100: J(s) = -8.9978
Iter   200: J(s) = -8.9352
Iter   500: J(s) = -8.9352   (stagnant)
Iter  1000: J(s) = -8.9352   (best found)

=== Simulated Annealing Convergence (max_iter=1000) ===
T=10.000  Iter    10: J(s) = -9.0124  best=-9.0124
T= 6.083  Iter    50: J(s) = -7.8412  best=-7.8412
T= 2.509  Iter   200: J(s) = -5.9821  best=-5.9821
T= 0.897  Iter   500: J(s) = -5.1835  best=-5.1823
T= 0.033  Iter  1000: J(s) = -5.1826  best=-5.1823  (SA TERMINATED)
```

Contoh 7. Perbandingan Akhir Tiga Algoritma

Output (Ringkasan Perbandingan)

```
======================================================================
           J(s)     RMSD_pi    #Eval    Waktu(s)  Feasible  pi_T
----------------------------------------------------------------------
HC        -8.9352    0.476      523      2.14      YES     2.55%
SA        -5.1823    0.241     1000      4.08      YES     2.48%
GA        -4.9571    0.218    10000*     5.73      YES     2.47%
----------------------------------------------------------------------
*GA: 50 individu x 200 generasi

Best J(s):        Genetic Algorithm (-4.9571)
Best RMSD_pi:     Genetic Algorithm (0.218)
Interpretasi:     0.218 < 0.50 -> SANGAT BAIK
```

Bab 3

Spesifikasi Bonus

3.1 Interaksi Kebijakan Fiskal-Moneter (Stackelberg Game)

Pada spesifikasi utama, program mengasumsikan Bank Indonesia beroperasi secara independen dalam menentukan lintasan BI-Rate, tanpa memperhitungkan respons strategis dari otoritas fiskal. Dalam praktiknya, keputusan suku bunga tidak dapat dipisahkan dari kebijakan fiskal pemerintah. Defisit APBN yang besar memerlukan penerbitan Surat Berharga Negara (SBN) dalam jumlah besar, yang pada gilirannya memengaruhi imbal hasil obligasi dan ekspektasi inflasi. Sebaliknya, suku bunga yang tinggi akan memberatkan biaya pembiayaan utang pemerintah. Interaksi dua arah ini telah menjadi perhatian sentral dalam literatur ekonomi moneter sejak Sargent dan Wallace [6] memperkenalkan konsep *unpleasant monetarist arithmetic* dan Leeper [7] memformalkan interaksi fiskal-moneter dalam kerangka rezim *active/passive*.

Bonus ini memodelkan interaksi strategis antara BI (otoritas moneter) dan Kementerian Keuangan (otoritas fiskal) sebagai *game* Stackelberg dua pemain. Dalam struktur Stackelberg, otoritas fiskal bertindak sebagai *leader* yang menetapkan lintasan defisit terlebih dahulu, dan BI bertindak sebagai *follower* yang mengoptimalkan lintasan suku bunga dengan mempertimbangkan defisit yang sudah ditetapkan. Urutan ini mencerminkan realitas kelembagaan Indonesia, di mana APBN disahkan DPR sebelum RDG BI menetapkan suku bunga untuk kuartal berjalan.

3.1.1 Perluasan State

State tidak lagi hanya berisi vektor suku bunga, melainkan diperluas menjadi pasangan state moneter dan fiskal.

$$s = \Big(\underbrace{(r_1, r_2, \ldots, r_T)}_{\text{BI (moneter)}}, \underbrace{(g_1, g_2, \ldots, g_T)}_{\text{Kemenkeu (fiskal)}} \Big)$$

Variabel $g_t \in \{-3{,}00\%, -2{,}75\%, \ldots, 0{,}00\%\}$ adalah rasio defisit fiskal terhadap PDB pada kuartal ke-t, didiskritisasi dalam kelipatan 25 basis poin. Tanda negatif menunjukkan defisit (pengeluaran melebihi penerimaan), dan nol menunjukkan anggaran berimbang. Batas defisit mengikuti ketentuan UU Keuangan Negara yang membatasi defisit maksimum 3% PDB per tahun.

3.1.2 Perluasan Model Makroekonomi

Persamaan IS Curve diperluas dengan menambahkan kanal stimulus fiskal. Defisit pemerintah pada periode $t-1$ memberikan dorongan tambahan terhadap *output gap* pada periode t karena belanja pemerintah langsung meningkatkan permintaan agregat.

$$y_t = \rho_y \, y_{t-1} - \beta(r_{t-1} - \pi_{t-1} - r^*) + \gamma_g \, g_{t-1} + \epsilon_t^{IS}$$

Parameter baru γ_g mengukur pengganda fiskal (*fiscal multiplier*), yaitu seberapa besar satu poin persentase defisit terhadap PDB mampu mendorong *output gap*. Nilai $\gamma_g = 0{,}50$ ditetapkan untuk keperluan simulasi

kelas. Persamaan Phillips Curve, UIP, dan proksi lainnya tidak berubah karena kanal transmisi fiskal ke inflasi dan nilai tukar sudah tertangkap melalui dampaknya terhadap *output gap*.

Otoritas fiskal memiliki fungsi objektif sendiri yang berbeda dari BI. Kementerian Keuangan bertujuan mendorong pertumbuhan ekonomi sambil menjaga defisit tetap terkendali.

$$L_{fiskal}(s) = \sum_{t=1}^{T} \Big[- w_y^f \, y_t + w_g \, g_t^2 \Big]$$

Suku pertama mendorong *output gap* setinggi mungkin (tanda negatif karena L_{fiskal} diminimalkan), sedangkan suku kedua menghukum defisit yang terlalu besar. Bobot $w_y^f = 1{,}0$ dan $w_g = 0{,}5$ ditetapkan sebagai default.

3.1.3 Mekanisme Pencarian *Joint Equilibrium*

Proses evaluasi satu state s kini memerlukan dua tahap. Pertama, otoritas fiskal sebagai *leader* menetapkan $g = (g_1, \ldots, g_T)$. Kedua, BI sebagai *follower* menjalankan *local search* sendiri untuk menemukan $r = (r_1, \ldots, r_T)$ yang memaksimalkan $J(s)$ dengan g yang sudah *given*. Hasil dari *nested search* BI ini menentukan *best response* terhadap g, dan nilai $J(s)$ dari *best response* tersebut menjadi skor yang digunakan untuk mengevaluasi state gabungan (r, g) pada level *leader*.

Secara implementatif, fungsi `compute_objective(s)` pada spesifikasi utama diganti dengan `compute_joint_equilibrium(g)` yang menerima lintasan fiskal, lalu menjalankan `simulated_annealing` atau `genetic_algorithm` untuk mencari r optimal terhadap g tersebut, dan mengembalikan skor $J(r, g)$ terbaik. Algoritma *local search* pada level *leader* (untuk mencari g optimal) tetap dapat menggunakan `hill_climbing`, `simulated_annealing`, atau `genetic_algorithm`, dengan modifikasi bahwa setiap evaluasi neighbor kini memanggil `compute_joint_equilibrium` yang di dalamnya menjalankan *nested search*.

3.2 Visualisasi *Local Search*

Pada spesifikasi wajib, program hanya menampilkan hasil akhir algoritma melalui antarmuka teks. Melalui bonus ini, anda akan ditantang untuk membuat sebuah GUI interaktif yang dapat memvisualisasikan proses pencarian ketiga algoritma secara *real-time* dan juga *side-by-side*. Fitur minimal yang harus tersedia pada GUI mencakup lima komponen.

1. Pertama, grafik konvergensi yang menampilkan nilai $J(s)$ terhadap iterasi untuk Hill-Climbing, Simulated Annealing, dan Genetic Algorithm dalam satu *chart* sehingga perbandingan kecepatan dan kualitas konvergensi ketiganya terlihat langsung.
2. Kedua, visualisasi state yang menampilkan plot r_t terhadap waktu untuk kandidat terbaik di tiap iterasi, dianimasikan agar terlihat bagaimana lintasan suku bunga berevolusi selama proses pencarian.
3. Ketiga, panel status *constraint* yang menampilkan indikator visual (hijau untuk terpenuhi, merah untuk dilanggar) untuk C1 sampai C5 pada kandidat solusi saat ini.
4. Keempat, kontrol *hyperparameter* interaktif berupa *slider* untuk mengubah *cooling schedule* SA, *mutation rate* GA, atau bobot w_π, w_y, w_{pp}, w_r pada fungsi objektif, yang memungkinkan pengguna menjalankan ulang pencarian dengan parameter berbeda tanpa menyentuh kode.
5. Kelima, mode perbandingan yang menjalankan ketiga algoritma dari *initial state* yang sama dan menampilkan solusi akhir masing-masing beserta nilai $J(s)$, status *feasible*, dan inflasi akhir π_T dalam satu tabel ringkasan.

Bibliografi

[1] J. Galí and T. Monacelli, "Monetary policy and exchange rate volatility in a small open economy," *Review of Economic Studies*, vol. 72, no. 3, pp. 707–734, 2005.

[2] R. Clarida, J. Galí, and M. Gertler, "The science of monetary policy: A new keynesian perspective," *Journal of Economic Literature*, vol. 37, no. 4, pp. 1661–1707, 1999.

[3] S. M. Juhro, "Kurva phillips dan perubahan struktural di indonesia: Keberadaan, pola pembentukan ekspektasi, dan linieritas," *Buletin Ekonomi Moneter dan Perbankan*, vol. 6, no. 4, pp. 41–76, 2004.

[4] M. Woodford, *Interest and Prices: Foundations of a Theory of Monetary Policy*. Princeton: Princeton University Press, 2003.

[5] L. E. O. Svensson, "Inflation forecast targeting: Implementing and monitoring inflation targets," *European Economic Review*, vol. 41, no. 6, pp. 1111–1146, 1997.

[6] T. J. Sargent and N. Wallace, "Some unpleasant monetarist arithmetic," *Federal Reserve Bank of Minneapolis Quarterly Review*, vol. 5, no. 3, pp. 1–17, 1981.

[7] E. M. Leeper, "Equilibria under 'active' and 'passive' monetary and fiscal policies," *Journal of Monetary Economics*, vol. 27, no. 1, pp. 129–147, 1991.

Loan Acceptance Prediction Technical Write-Up

Muhammad Aufar Rizqi Kusuma
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
13524061@std.stei.itb.ac.id

1 TL;DR

Tiga algoritma klasifikasi diimplementasikan *from-scratch*menggunakan `numpy` untuk prediksi status persetujuan pinjaman. CART dengan Gini impurity, `max_depth=19`, `min_samples_leaf=8`, dan `min_samples_split=100` mencapai Macro F1 tertinggi pada validasi silang maupun public leaderboard. Logistic Regression dan Linear SVM tertinggal signifikan karena batas keputusan non-linier pada dataset, meskipun keduanya telah menggunakan algoritma optimasi Adam sebagai bonus implementasi di luar materi kelas. Berbagai pendekatan seperti pruning, rekayasa fitur, kriteria splitting alternatif, dan manipulasi data diuji namun tidak memberikan peningkatan terhadap CART. Pengembangan selanjutnya disarankan mengarah ke ensemble methods dan perbaikan encoding fitur kategorik karena CART tunggal telah mencapai titik jenuh pada konfigurasi saat ini.

2 Problem Overview

Permasalahan yang diselesaikan adalah prediksi status persetujuan pinjaman (`loan_status`) dengan dua kelas yaitu 0 untuk ditolak dan 1 untuk disetujui. Prediksi dilakukan berdasarkan data demografis, finansial, dan riwayat kredit masing-masing pemohon. Metrik evaluasi yang digunakan adalah **Macro F1-score** yang merupakan rata-rata F1-score kedua kelas tanpa memperhitungkan proporsi. Metrik ini dipilih karena distribusi kelas yang tidak seimbang sehingga model tidak bisa hanya mengandalkan prediksi kelas mayoritas.

Dataset terdiri dari `train.csv` dengan 36.000 baris dan `test.csv` dengan 9.000 baris tanpa missing values. Dari 36.000 sampel latih, sekitar 28.000 berlabel ditolak dan 8.000 berlabel disetujui, menghasilkan rasio ketidakseimbangan sekitar 3,5 banding 1. Terdapat 8 fitur numerik yaitu `person_age`, `person_income`, `person_emp_exp`, `loan_amnt`, `loan_int_rate`, `loan_percent_income`, `cb_person_cred_hist_length`, dan `credit_score`, serta 3 fitur kategorik yaitu `person_gender`, `person_home_ownership`, dan `previous_loan_defaults_on_file`. Kolom `person_id` tidak digunakan dalam prediksi dan target `loan_status` hanya tersedia pada data latih.

3 Decision Tree Learning (DTL)

Dari tiga varian yang diperbolehkan yaitu ID3 [2], C4.5 [3], dan CART [1], varian CART dipilih karena menangani fitur numerik dan kategorik secara langsung melalui binary split dengan Gini impurity, tanpa memerlukan diskretisasi eksplisit seperti ID3 maupun perhitungan information gain ratio seperti C4.5. CART membangun pohon biner dengan memilih fitur dan threshold yang memaksimalkan penurunan Gini impurity pada setiap simpul [1, 4]. Implementasi dibangun dari awal hanya menggunakan `numpy`. Fitur kategorik diubah menjadi integer encoding lalu seluruh fitur distandardisasi dengan Z-score.

Eksperimen dimulai dari baseline sederhana dengan `max_depth=10` dan `min_samples_leaf=5` yang hanya memperoleh Macro F1 sekitar 0,86 pada validasi silang. Observasi awal menunjukkan bahwa pohon yang lebih dalam memberikan hasil yang lebih baik karena dataset memiliki batas keputusan non-linier yang kompleks, sehingga kedalaman dinaikkan secara bertahap. Pada `max_depth=19` dan `min_samples_leaf=7`, skor validasi silang meningkat ke sekitar 0,87, menandakan bahwa kedalaman yang lebih besar memungkinkan pohon menangkap lebih banyak interaksi antar fitur.

Langkah berikutnya adalah ablasi fitur untuk mengidentifikasi fitur yang memberikan kontribusi positif. Setiap fitur dihapus satu per satu dan model dilatih ulang dengan konfigurasi yang sama, lalu dievaluasi pada validasi silang. Hasilnya menunjukkan bahwa menghapus `person_age` memberikan peningkatan skor yang konsisten, sementara menghapus fitur lain justru menurunkan skor. Investigasi lebih lanjut mengungkapkan bahwa `person_age` mengandung outlier ekstrem seperti 144 tahun yang menciptakan split tidak informatif di area tertentu pada pohon. Temuan ini mendorong pendekatan alternatif yaitu melakukan capping pada nilai 100 tahun alih-alih menghapus fitur sepenuhnya. Dengan capping, `person_age` justru memberikan kontribusi positif karena informasi usia yang valid tetap dipertahankan sementara noise dari outlier dihilangkan.

Eksperimen regularisasi kemudian mengeksplorasi penambahan `min_samples_split` sebagai stopping criterion ketiga, melengkapi `max_depth` dan `min_samples_leaf` yang sudah ada. Parameter ini mencegah pemisahan pada simpul yang jumlah sampelnya di bawah threshold tertentu, berbeda dengan `min_samples_leaf` yang membatasi ukuran anak setelah pemisahan. Nilai `min_samples_split=100` ditemukan optimal melalui grid search pada validasi silang, memberikan peningkatan sekitar 0,001 dibandingkan tanpa parameter ini. Nilai yang lebih besar dari 200 justru menurunkan performa karena terlalu banyak membatasi pertumbuhan pohon.

Sebagai bonus eksplorasi, Bayesian Optimization (BO) diimplementasikan sebagai alternatif grid search untuk pencarian hyperparameter. BO memodelkan fungsi objektif $f(\theta)$ yang tidak diketahui, dalam hal ini Macro F1 sebagai fungsi dari $(\text{max_depth}, \text{min_samples_leaf}, \text{min_samples_split})$, menggunakan Gaussian Process (GP) sebagai surrogate model [5, 6]. GP memberikan distribusi posterior dengan mean $\mu(\theta)$ dan varians $\sigma^2(\theta)$ untuk setiap titik kandidat berdasarkan observasi sebelumnya. Kernel yang digunakan adalah Radial Basis Function $k(\theta_i, \theta_j) = \exp(-\|\theta_i - \theta_j\|^2/2\ell^2)$ dengan length scale ℓ yang mengontrol smoothness fungsi [6]. Akuisisi dilakukan menggunakan Expected Improvement [5]

$$\text{EI}(\theta) = (\mu(\theta) - f^* - \xi)\,\Phi(Z) + \sigma(\theta)\,\phi(Z), \quad Z = \frac{\mu(\theta) - f^* - \xi}{\sigma(\theta)}$$

di mana f^* adalah nilai terbaik yang telah diamati dan ξ adalah parameter eksplorasi. BO dijalankan selama 25 iterasi dengan 5 sampel awal acak, hanya menggunakan 30 evaluasi total dibandingkan ribuan kombinasi pada grid search. Hasil BO menemukan konfigurasi `max_depth=29`, `min_samples_leaf=12`, `min_samples_split=97` dengan skor validasi silang 0,8692 yang lebih tinggi dari konfigurasi grid search 0,8676. Namun ketika dievaluasi pada data uji, konfigurasi BO justru memberikan skor yang lebih rendah. Ini menegaskan, bahwa validasi silang lokal tidak selalu berkorelasi sempurna dengan performa pada data uji.

Konfigurasi akhir yang digunakan adalah `max_depth=19`, `min_samples_leaf=8`, dan `min_samples_split=100` dengan seluruh 11 fitur termasuk `person_age` yang sudah di-cap pada 100 tahun. Konfigurasi ini memperoleh Macro F1 0,8676 pada validasi silang dan 0,87076 pada Kaggle public leaderboard. Model CART from-scratch dibandingkan dengan `DecisionTreeClassifier` dari scikit-learn menggunakan konfigurasi hyperparameter yang sama dan menghasilkan prediksi yang identik, mengonfirmasi kebenaran implementasi.

Sebagai bonus, pohon keputusan juga divisualisasikan pada Gambar 1 yang menampilkan empat level teratas dengan split condition, Gini impurity, jumlah sampel, dan distribusi kelas pada setiap simpul. Warna hijau menandakan simpul dengan mayoritas kelas 1 (disetujui) dan merah menandakan mayoritas kelas 0 (ditolak). Simpul pertama langsung memisahkan berdasarkan `previous_loan_defaults_on_file`, mengonfirmasi bahwa fitur ini adalah pemisah paling informatif dengan seluruh sampel yang memiliki riwayat gagal bayar langsung diarahkan ke cabang penolakan. Level berikutnya menunjukkan bahwa pada subset tanpa riwayat gagal bayar, `loan_percent_income` dan `loan_int_rate` menjadi pemisah dominan, konsisten dengan temuan bahwa rasio pinjaman terhadap pendapatan dan suku bunga adalah pembeda paling kuat antar kelas. Struktur pohon yang tidak simetris juga mencerminkan ketidakseimbangan dataset dengan cabang penolakan yang lebih padat sampel dibandingkan cabang persetujuan.

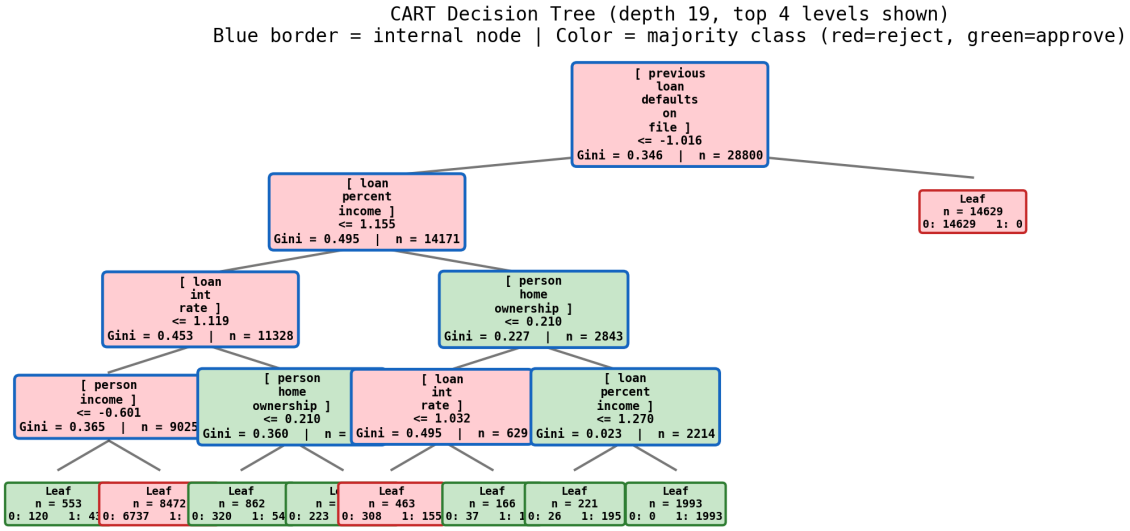


Figure 1: Visualisasi empat level teratas pohon CART. Simpul internal (border biru) menampilkan fitur dan threshold split. Warna hijau menunjukkan mayoritas kelas 1 (disetujui) dan merah menunjukkan mayoritas kelas 0 (ditolak).

4 Logistic Regression (LR)

Logistic Regression diimplementasikan sebagai model linier dengan fungsi sigmoid untuk menghasilkan probabilitas kelas. Model dilatih menggunakan Binary Cross-Entropy loss dengan tambahan regularisasi L2 untuk mencegah overfitting. Opti-

masi dilakukan dengan algoritma Adam yang menggabungkan momentum dan adaptive learning rate, memberikan konvergensi yang lebih cepat dan stabil dibandingkan Stochastic Gradient Descent biasa mengingat skala fitur yang cukup bervariasi pada dataset ini.

Untuk menangani ketidakseimbangan kelas, diterapkan class weighting yang memberikan bobot lebih besar pada sampel kelas minoritas. Bobot dihitung secara otomatis berdasarkan proporsi kelas (`class_weight="balanced"`) sehingga model tidak bias ke kelas mayoritas. Setelah pelatihan, dilakukan threshold optimization untuk mencari ambang batas probabilitas yang memaksimalkan Macro F1 pada data validasi, karena threshold default 0,5 cenderung suboptimal pada data tidak seimbang.

Hasil Logistic Regression from-scratch memperoleh Macro F1 sekitar 0,83 pada validasi silang, dikonfirmasi konsisten dengan implementasi `LogisticRegression` dari scikit-learn. Skor ini jauh di bawah CART yang mencapai 0,87 pada validasi yang sama, menegaskan bahwa batas keputusan pada dataset ini bersifat non-linier dan model linier tidak mampu menangkap pola pemisahan antar kelas secara memadai.

Sebagai bonus, proses training divisualisasikan pada Gambar 2. Kurva loss menunjukkan penurunan cepat pada 100 iterasi pertama yang kemudian melambat secara gradual dan berhenti pada iterasi ke-675 karena perubahan loss sudah di bawah toleransi 10^{-6}. Skala logaritmik memperlihatkan bahwa penurunan terbesar terjadi di awal, konsisten dengan perilaku Adam yang cepat keluar dari daerah gradien besar menuju lembah minimum yang lebih datar. Kontur loss pada dua dimensi bobot paling dominan menunjukkan lintasan parameter yang dimulai dari titik inisialisasi nol (hijau) dan bergerak menuju minimum lokal (merah). Lintasan tidak lurus melainkan melengkung, menunjukkan adanya korelasi antar parameter yang membuat arah gradien tidak sejajar dengan arah menuju minimum. Optimizer Adam mampu menavigasi kelengkungan ini dengan efektif, mencapai konvergensi dalam waktu yang relatif singkat.

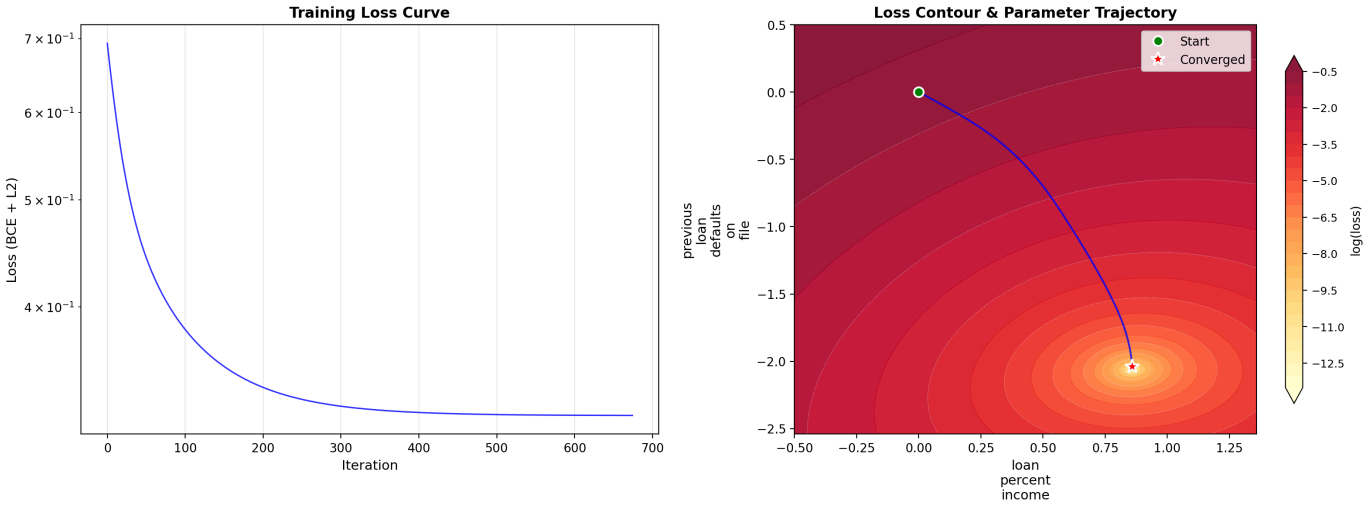


Figure 2: Kiri: kurva penurunan Binary Cross-Entropy loss selama training dengan early stopping pada iterasi ke-675. Kanan: kontur loss pada dua dimensi bobot paling dominan (`loan_percent_income` dan `previous_loan_defaults_on_file`) beserta lintasan parameter dari inisialisasi (hijau) menuju konvergensi (merah).

5 Support Vector Machine (SVM)

Linear SVM diimplementasikan dengan hinge loss dan regularisasi L2. Model mencari hyperplane yang memaksimalkan margin antar kelas melalui parameter regularisasi C yang mengontrol trade-off antara lebar margin dan penalti kesalahan klasifikasi. Optimasi menggunakan algoritma Adam dengan class weighting untuk menangani ketidakseimbangan data, serupa dengan pendekatan pada Logistic Regression.

Prediksi SVM didasarkan pada nilai decision function yang dihitung dari hasil dot product bobot dan fitur ditambah bias. Threshold default adalah nol, namun dilakukan optimasi threshold pada data validasi untuk menemukan nilai yang memaksimalkan Macro F1, mengingat distribusi kelas yang tidak seimbang dapat menggeser titik keputusan optimal.

SVM from-scratch memperoleh Macro F1 sekitar 0,81 pada validasi silang, hasil yang konsisten dengan `LinearSVC` dari scikit-learn. Skor ini merupakan yang terendah di antara ketiga algoritma, sedikit di bawah Logistic Regression dan cukup jauh di bawah CART. Keterbatasan model linier dalam menangkap batas keputusan non-linier menjadi penyebab utama rendahnya performa SVM maupun Logistic Regression pada dataset ini.

6 Algoritma Optimasi Adam

Baik Logistic Regression maupun SVM dilatih menggunakan algoritma Adam sebagai pengganti Stochastic Gradient Descent (SGD). Adam, yang diperkenalkan oleh Kingma dan Ba [7], menggabungkan dua ide utama yaitu momentum melalui estimasi momen pertama $m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla f(\theta_t)$ dan adaptive learning rate melalui estimasi momen kedua $v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla f(\theta_t))^2$. Parameter diperbarui dengan aturan $\theta_{t+1} = \theta_t - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ di mana $\hat{m}_t$ dan $\hat{v}_t$ adalah versi bias-corrected dari m_t dan v_t.

Bias correction menjadi komponen penting yang membedakan Adam dari optimizer adaptif lain seperti RMSprop [7]. Pada iterasi awal, estimasi m_t dan v_t cenderung bias menuju nol karena inisialisasi dari nol. Adam mengoreksi bias ini melalui $\hat{m}_t = m_t/(1-\beta_1^t)$ dan $\hat{v}_t = v_t/(1-\beta_2^t)$, memastikan learning rate yang efektif tidak terlalu besar di awal training.

Pada dataset ini, Adam dipilih karena dua alasan. Pertama, skala fitur yang sangat bervariasi menyebabkan beberapa parameter memiliki gradien dengan magnitudo jauh lebih besar dibanding yang lain, dan adaptive learning rate membantu menyeimbangkan langkah pembaruan di seluruh dimensi. Kedua, lanskap loss yang asimetris akibat regularisasi L2 dan class weighting menciptakan kelengkungan yang bervariasi, dan momentum membantu navigasi melewati daerah datar menuju minimum. Tanpa Adam, SGD standar memerlukan tuning learning rate yang jauh lebih sensitif dan jumlah iterasi yang lebih besar untuk mencapai konvergensi yang setara. Keunggulan Adam dalam menangani permasalahan non-convex dan noisy gradients telah didokumentasikan secara luas, termasuk oleh Ruder [8] yang membandingkan berbagai varian gradient descent secara komprehensif.

Visualisasi lintasan parameter pada kontur loss yang telah ditampilkan sebelumnya mengonfirmasi efektivitas Adam. Lintasan dari inisialisasi menuju konvergensi tidak lurus melainkan melengkung mengikuti kontur loss, dan Adam mampu menavigasi kelengkungan ini dalam 675 iterasi sebelum early stopping terpicu. Implementasi Adam from-scratch diverifikasi menghasilkan konvergensi yang identik dengan implementasi referensi pada library standar.

7 Validation Strategy

Validasi dilakukan dengan stratified 5-fold cross-validation pada data latih. Stratifikasi menjaga proporsi kelas di setiap fold sehingga distribusi tetap mencerminkan data asli. Setiap model dilatih pada empat fold dan dievaluasi pada satu fold secara bergantian, kemudian skor Macro F1 dirata-ratakan untuk memperoleh estimasi performa yang stabil.

Table 1: Hasil validasi dan perbandingan dengan Kaggle public leaderboard.

Model	**CV Mean**	**CV Std**	**Kaggle LB**
CART	0.8676	$\pm$0.0072	0.87076
Logistic Regression	0.8062	$\pm$0.0030	–
SVM	0.7670	$\pm$0.0060	–

CART memperoleh skor validasi tertinggi di antara ketiga model, dengan deviasi standar yang relatif kecil mengindikasikan stabilitas yang baik antar fold. Skor Kaggle public leaderboard untuk CART sedikit lebih tinggi dibanding rata-rata cross-validation, perbedaan sekitar 0,003 yang masih dalam rentang satu deviasi standar. Kesenjangan ini wajar mengingat public leaderboard hanya mengevaluasi subset dari data uji. Logistic Regression dan SVM tidak disubmit ke Kaggle karena skor validasinya sudah cukup jauh di bawah CART sehingga kecil kemungkinan memberikan perbaikan yang berarti.

8 Percobaan yang Gagal

Sejumlah pendekatan diujicobakan pada model CART dan dievaluasi menggunakan validasi silang. Tidak satu pun memberikan peningkatan terhadap Macro F1, bahkan sebagian besar justru menurunkan performa.

Pruning. CCP (Cost-Complexity Pruning) dan F1-based pruning diimplementasikan untuk mengurangi overfitting dengan memangkas cabang yang tidak signifikan. Secara teoretis, pruning seharusnya meningkatkan generalisasi dengan menyederhanakan pohon dan menghilangkan simpul yang terlalu spesifik terhadap data latih. Pada dataset ini, kedua metode pruning justru menurunkan Macro F1 pada validasi silang secara konsisten sekitar 0,01. Fenomena ini menunjukkan bahwa kompleksitas yang dihasilkan oleh stopping criteria berbasis `max_depth` dan `min_samples_leaf` sudah cukup optimal, dan pruning tambahan hanya menghilangkan cabang yang sebenarnya informatif. Pohon pada konfigurasi akhir memiliki kedalaman yang moderat dengan regularisasi yang memadai sehingga pemangkasan lebih lanjut bersifat destruktif.

Rekayasa fitur. Berbagai pendekatan rekayasa fitur diuji seperti binary flags untuk kondisi spesifik, fitur interaksi antar variabel, dan transformasi logaritmik serta polinomial. Seluruhnya gagal meningkatkan skor validasi silang, dengan penurunan berkisar 0,005 hingga 0,01 dari baseline. CART secara inheren sudah menangkap interaksi antar fitur melalui sequential binary split sehingga penambahan fitur buatan justru menambah dimensi tanpa memberikan informasi baru yang tidak dapat ditemukan oleh pohon secara mandiri.

Kriteria splitting alternatif. Twoing criterion diuji sebagai alternatif Gini impurity namun memberikan hasil yang netral pada validasi silang, tidak menaikkan maupun menurunkan skor secara berarti. C4.5 dengan Information Gain Ratio juga diimplementasikan sebagai perbandingan. Information Gain Ratio membagi information gain dengan split information untuk menghukum pemisahan yang terlalu tidak seimbang, namun pada dataset dengan ketidakseimbangan kelas alami seperti ini, pemisahan yang informatif justru seringkali tidak seimbang. IGR secara konsisten menghasilkan skor validasi yang jauh lebih rendah, sekitar 0,02 di bawah Gini, karena terlalu agresif menghukum split yang memisahkan mayoritas dari minoritas.

Manipulasi data. Undersampling kelas mayoritas untuk menyeimbangkan dataset (balanced undersampling) menurunkan skor validasi secara drastis menjadi sekitar 0,82. Kehilangan sekitar 20.000 sampel kelas mayoritas menghilangkan terlalu banyak informasi tentang karakteristik pemohon yang ditolak. Class weighting sebagai alternatif yang lebih halus juga tidak membantu, menurunkan skor validasi sekitar 0,01 karena mengubah estimasi probabilitas di daun dan menggeser batas keputusan ke arah yang kurang optimal. Bootstrap model selection, yaitu melatih banyak pohon pada sampel bootstrap yang berbeda dan memilih yang terbaik, juga tidak berhasil karena setiap pohon dilatih pada subset data yang kurang representatif.

Grow-deep lalu post-prune. Strategi menumbuhkan pohon sangat dalam tanpa batasan lalu melakukan pemangkasan bottom-up berdasarkan `min_samples_leaf` diuji sebagai alternatif dari pemberian batasan di depan. Hasil validasi silang menunjukkan skor yang identik dengan pendekatan konvensional, menandakan bahwa urutan penerapan batasan tidak mengubah pohon akhir selama nilai parameternya ekuivalen.

9 Pengembangan Lebih Lanjut

Eksperimen menunjukkan bahwa CART tunggal telah mencapai titik jenuh pada konfigurasi saat ini dan peningkatan lebih lanjut memerlukan perubahan paradigma. Ensemble methods seperti Random Forest dan Gradient Boosting merupakan arah yang paling menjanjikan karena mampu mengagregasi banyak pohon lemah menjadi prediktor yang lebih kuat dan stabil, terutama pada batas keputusan non-linier yang tidak dapat ditangkap oleh satu pohon.

Dari sisi preprocessing, target encoding untuk fitur kategorik dapat menggantikan integer encoding saat ini. Target encoding mengganti setiap kategori dengan rata-rata target pada kategori tersebut sehingga memberikan informasi yang lebih kaya dibandingkan integer encoding yang bersifat arbitrer. Pendekatan ini perlu dilakukan dengan hati-hati melalui cross-validation untuk menghindari data leakage.

Regularisasi berbasis `min_impurity_decrease` juga layak dieksplorasi sebagai stopping criterion tambahan. Parameter ini mencegah pemisahan yang tidak memberikan penurunan impurity yang signifikan, berbeda dengan `min_samples_split` yang hanya melihat ukuran simpul. Kombinasi keduanya dapat menghasilkan pohon yang lebih ramping tanpa mengorbankan akurasi.

References

[1] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[2] J. R. Quinlan, "Induction of Decision Trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[3] J. R. Quinlan, *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.

[4] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.

[5] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," in *NeurIPS*, 2012.

[6] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[7] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *ICLR*, 2015.

[8] S. Ruder, "An Overview of Gradient Descent Optimization Algorithms," *arXiv preprint arXiv:1609.04747*, 2016.

[9] GeeksforGeeks, "Iterative Dichotomiser 3 (ID3) Algorithm From Scratch." https://www.geeksforgeeks.org/iterative-dichotomiser-3-id3-algorithm-from-scratch/

[10] GeeksforGeeks, "CART (Classification And Regression Tree) in Machine Learning." https://www.geeksforgeeks.org/cart-classification-and-regression-tree-in-machine-learning/

[11] GeeksforGeeks, "Understanding Logistic Regression." https://www.geeksforgeeks.org/understanding-logistic-regression/

[12] GeeksforGeeks, "Support Vector Machine (SVM) Algorithm." https://www.geeksforgeeks.org/support-vector-machine-algorithm/